

PassProof

Smart OD Management System

CS23333 – OBJECT ORIENTED PROGRAMMING USING JAVA PROJECT REPORT

Submitted by

MOHITH ROSHAN T

241001139

MUKESH RAJA V

241001140

OF

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM
(An Autonomous Institution)

Admin

HOD

Dept Head

Event Head

Faculty

Student

RAJALAKSHMI ENGINEERING COLLEGEDepartment of Information Technology

BONAFIDE CERTIFICATE

Certified that this project titled "**PassProof — Smart OD Management System**" is the bonafide work of **MOHITH ROSHAN T (241001139)** and **MUKESH RAJA V (241001140)**, students of B.Tech Information Technology, who carried out the project work under my supervision. This project is submitted in partial fulfilment of the requirements for the course **CS23333 – Object Oriented Programming Using Java**.

SIGNATURE**SIGNATURE****Dr. Valarmathie**

HEAD OF THE DEPARTMENT

Information Technology

Rajalakshmi Engineering College

Mrs. Usha S

COURSE INCHARGE

Information Technology

Rajalakshmi Engineering College

This project is submitted for **CS23333 – Object Oriented Programming Using Java** held on

INTERNAL EXAMINER**EXTERNAL EXAMINER**

TABLE OF CONTENTS

1	Abstract	4
2	Introduction	4
3	Purpose	5
4	Scope of the Project	5
5	Software Requirements Specification	6
	5.1 Hardware Requirements	6
	5.2 Software Requirements	6
	5.3 Functional Requirements	7
	5.4 Non-Functional Requirements	7
6	System Flow Diagrams	8
	6.1 System Architecture Diagram	8
	6.2 Use Case Diagram	9
	6.3 Entity-Relationship Diagram	10
	6.4 Data Flow Diagram (Level-1)	11
	6.5 Sequence Diagram	12
	6.6 Class Diagram	13
	6.7 Deployment Diagram	14
	6.8 OD Request Workflow Diagram	15
7	Module Description	16
8	MongoDB Collections & Schema	18
9	API Routes	19
10	Implementation — Screenshots	20
11	Backend Code Snippets	21
12	Security Features	23
13	Testing	24
14	Conclusion	25
15	References	25

1. Abstract

PassProof is a full-stack web-based On-Duty (OD) Management System built to address the shortcomings of manual OD attendance tracking in engineering colleges. The system introduces a structured, role-based digital workflow that eliminates fake OD claims, streamlines multi-level approvals, and provides verified OD Passes with QR-code authentication. Built with Python (Flask) as the backend and MongoDB as the database, it supports six distinct user roles — Admin, HOD, Department Head, Event Head, Faculty, and Student — each with clearly defined permissions and dedicated dashboards. The system handles two types of OD: Event OD (submitted by Event Heads for group participants) and Individual OD (applied personally by students for internships, medical visits, or other commitments). Both types undergo a two-level approval process — Department Head followed by HOD — ensuring institutional accountability at every stage.

2. Introduction

In college environments, attendance management is a critical administrative function. When students participate in events outside the classroom or require leave for personal academic obligations, they are granted OD (On-Duty) status. However, the current manual process suffers from several inherent weaknesses: teachers have no reliable way to verify OD claims in real time, Event Heads lack a systematic channel to submit student lists, and the absence of a digital audit trail makes fraud prevention difficult.

PassProof addresses these challenges by creating a centralized, multi-role OD management portal. Each stakeholder — from the Admin who manages user accounts, to the Faculty who verifies OD at the classroom level — has a dedicated, secure dashboard. The system integrates QR-code-based pass verification, bulk Excel uploads for user onboarding, automated conflict detection, and comprehensive audit logging to produce a robust, production-ready solution.

3. Purpose

- Eliminate fake or unverified OD claims in college attendance management.
- Provide a structured two-level approval workflow (Department Head → HOD) for all OD types.
- Auto-generate unique, QR-code-enabled OD Passes upon HOD approval.
- Allow Faculty to instantly verify student OD status by roll number or QR scan.
- Maintain a tamper-proof audit log of every action performed in the system.
- Simplify user onboarding for hundreds of students via bulk Excel upload.
- Support two distinct OD types: Event OD and Individual OD.

4. Scope of the Project

PassProof is designed for deployment in small to medium-sized engineering colleges with multiple departments. The system scope encompasses:

Role-Based Access Control (RBAC)	Six roles with granular permissions, JWT-secured sessions.
Dual OD Workflow	Event OD (group) and Individual OD (personal) with identical two-level approval.

OD Pass Generation	Unique Pass IDs in OD-YEAR-DEPT-XXXX format, downloadable as PDF with QR code.
Analytics & Reporting	Department-wise charts, event-wise pie charts, and exportable PDF/Excel reports.
Audit Trail	Full chronological log of every action, accessible to HOD.
Admin Onboarding	Bulk Excel upload for students and staff with auto-generated default credentials.
Scalability	MongoDB's document model allows effortless schema evolution as requirements grow.

5. Software Requirements Specification

5.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	4 GB (8 GB recommended)
Storage	50 GB free disk space
Network	Internet access (for MongoDB Atlas, optional)
OS	Windows 10/11, Ubuntu 20.04+, macOS 11+

5.2 Software Requirements

Layer	Technology	Version
Backend	Python + Flask	3.10+ / 3.x
Database	MongoDB	6.0+
Authentication	Flask-JWT-Extended	4.x
Password Hashing	bcrypt	4.x
Excel Processing	openpyxl + pandas	3.x / 2.x
PDF Generation	ReportLab	4.x
QR Code	qrcode[pil] + Pillow	7.x / 10.x
Frontend	HTML5, CSS3, JavaScript	ES6+
Charts	Chart.js	4.x
IDE	VS Code	Latest
DB GUI	MongoDB Compass	Latest

5.3 Functional Requirements

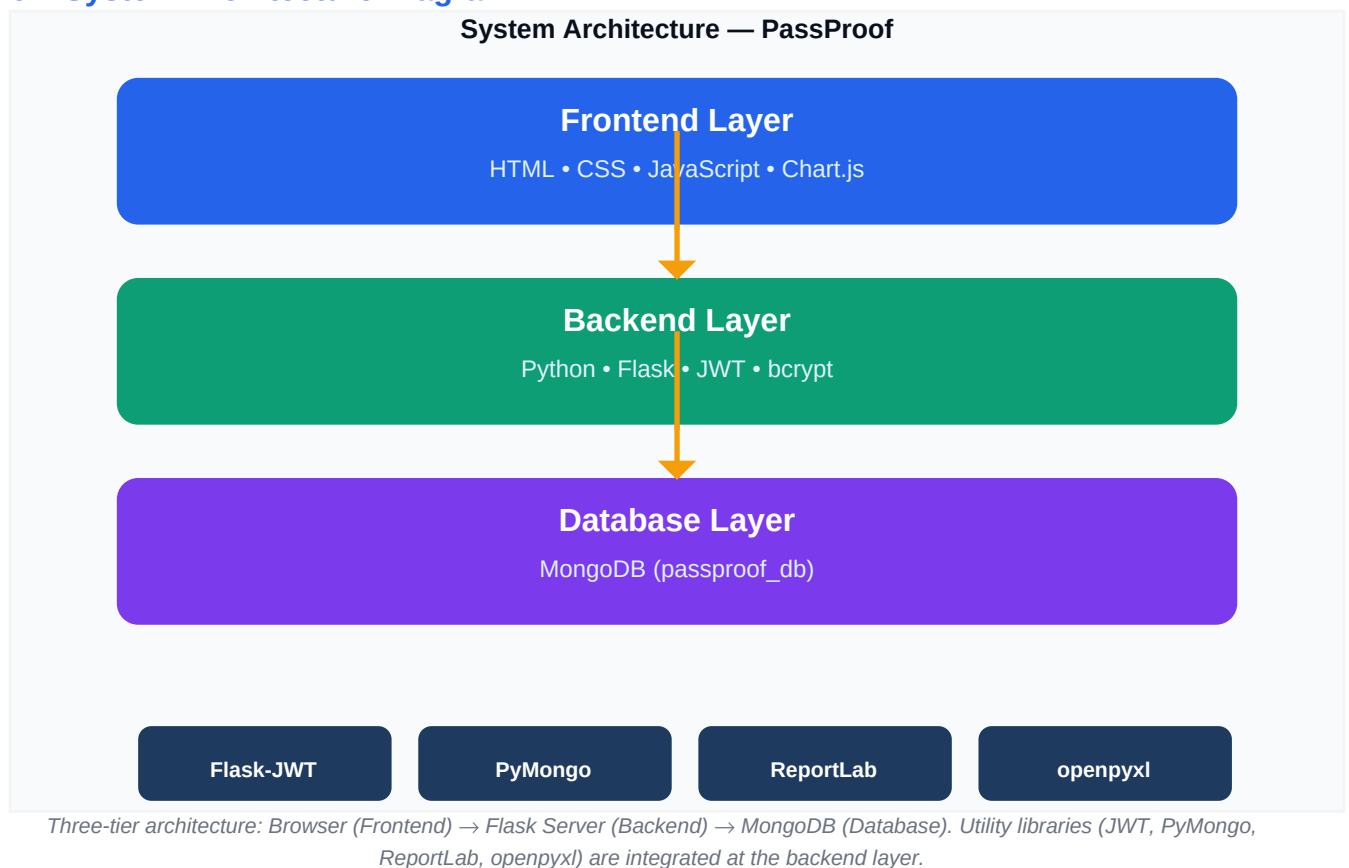
- Login Module: Role-auto-detect login using email/roll no/staff ID + bcrypt password.
- Admin Module: Create, edit, deactivate users; bulk upload via Excel with validation.
- Event OD Module: Event creation, student list submission, lock after approval.
- Individual OD Module: Student self-apply with supporting document upload.
- Department Head Module: First-level review, approve/reject with mandatory reason.
- HOD Module: Final approval, HOD letter upload, bulk approve, conflict alerts.
- Faculty Module: Roll number search, class-wise OD summary per period.
- Student Portal: OD status timeline, OD Pass download as PDF with QR code.
- Analytics: Charts for OD trends, department-wise stats, frequent OD students.
- Audit Log: Immutable timestamped log of every system action.
- Export: PDF and Excel reports with filter options.

5.4 Non-Functional Requirements

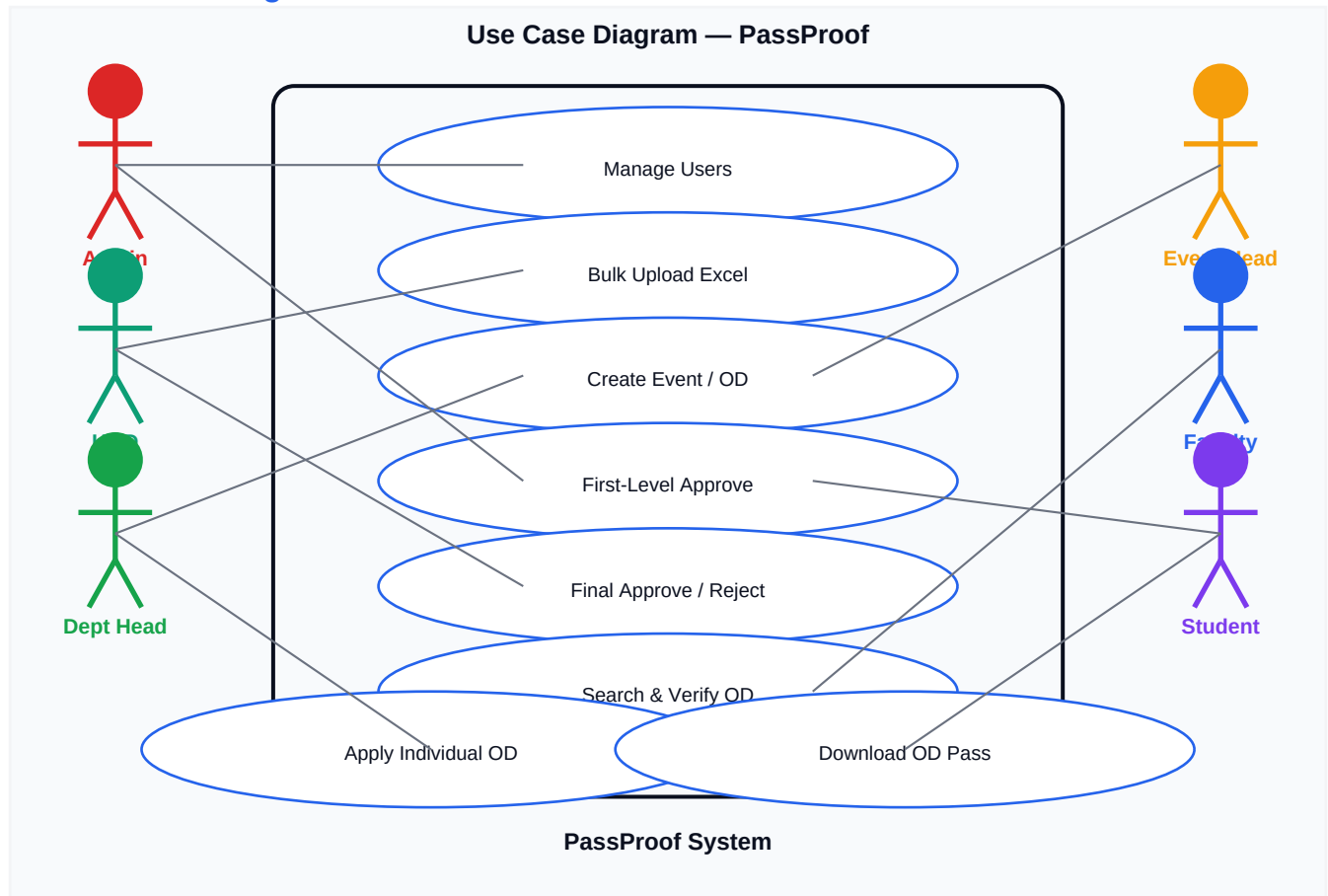
Property	Requirement
Security	JWT tokens (8h expiry), bcrypt hashing, role-based route guards
Performance	MongoDB indexed queries, paginated results (20/page)
Usability	Mobile-responsive UI, toast notifications, skeleton loaders
Reliability	Conflict detector prevents duplicate OD on same date
Maintainability	Modular Flask blueprints, separation of models/routes/utils
Auditability	Every CRUD action logged with user ID and timestamp

6. System Flow Diagrams

6.1 System Architecture Diagram

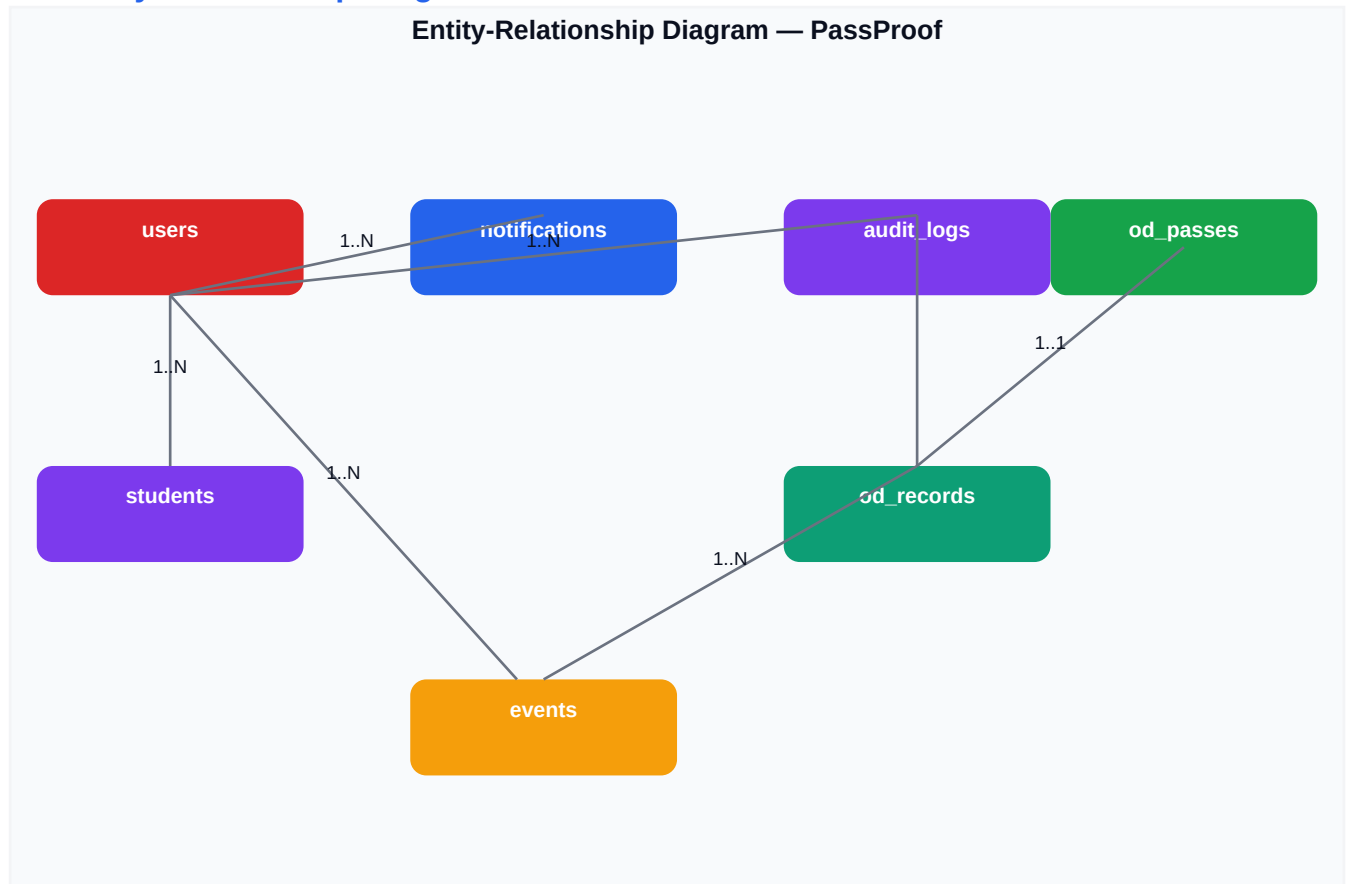


6.2 Use Case Diagram



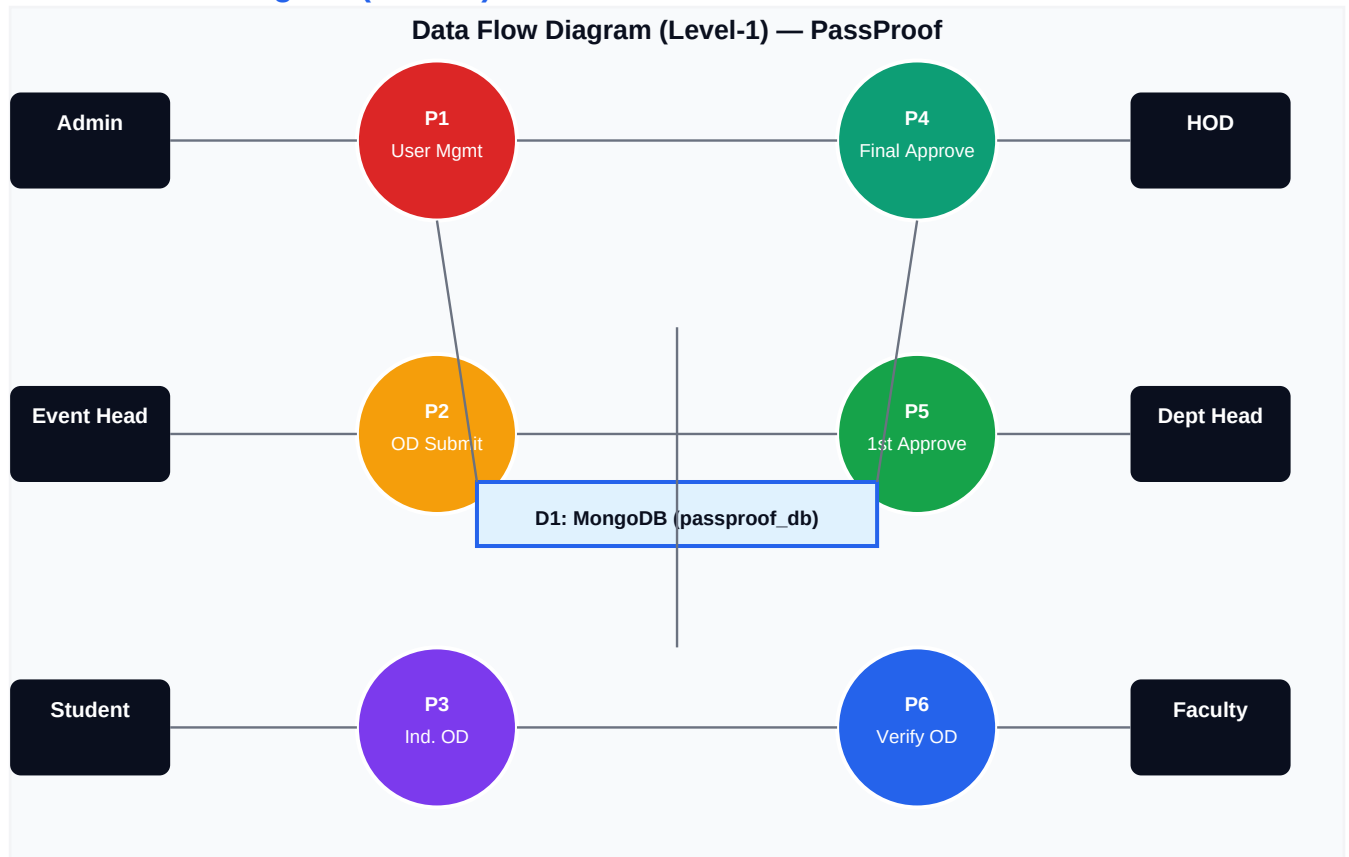
Six actors interact with the PassProof system. Admin manages users. Event Head submits OD lists. Department Head performs first-level approval. HOD performs final approval. Faculty verifies OD. Student applies Individual OD and downloads OD Pass.

6.3 Entity-Relationship Diagram



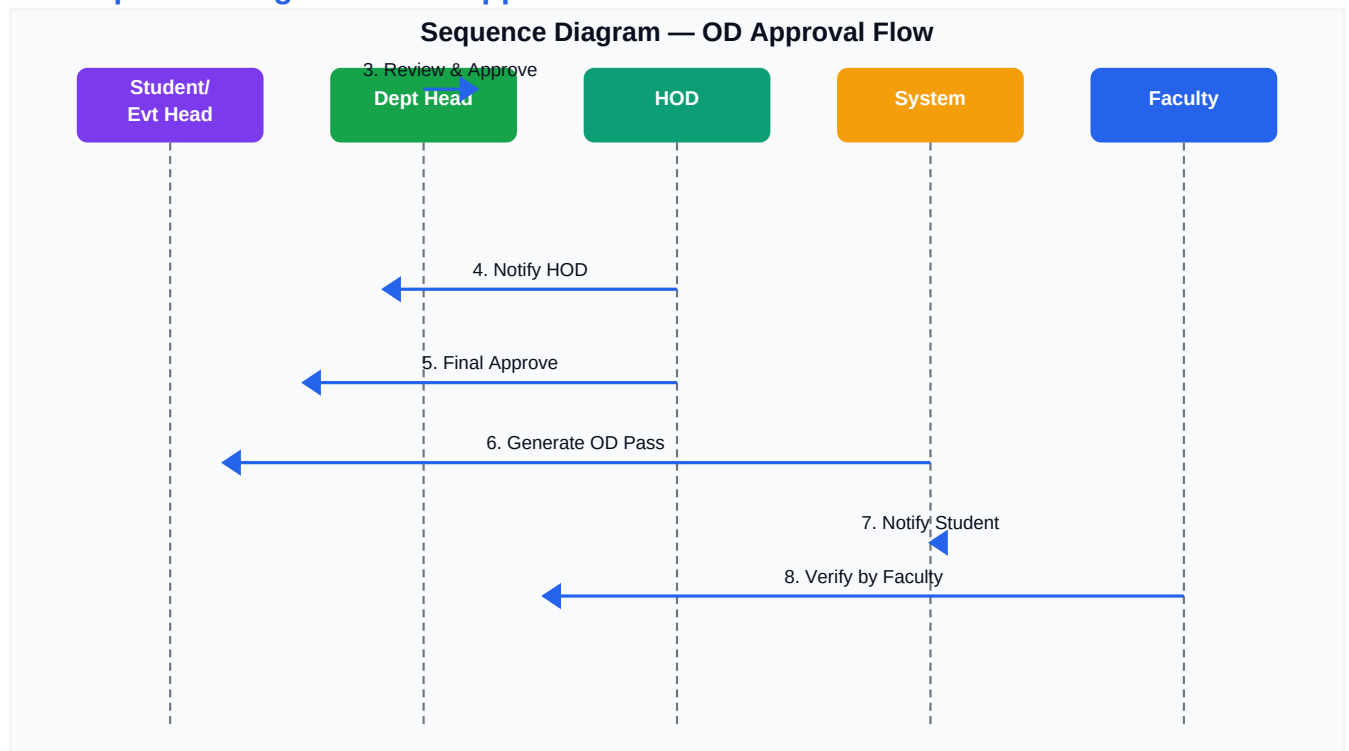
Seven MongoDB collections: *users*, *students*, *events*, *od_records*, *od_passes*, *notifications*, and *audit_logs*. *od_records* references both *students* and *events* (nullable for Individual OD). *od_passes* are generated from *od_records* upon HOD approval.

6.4 Data Flow Diagram (Level-1)



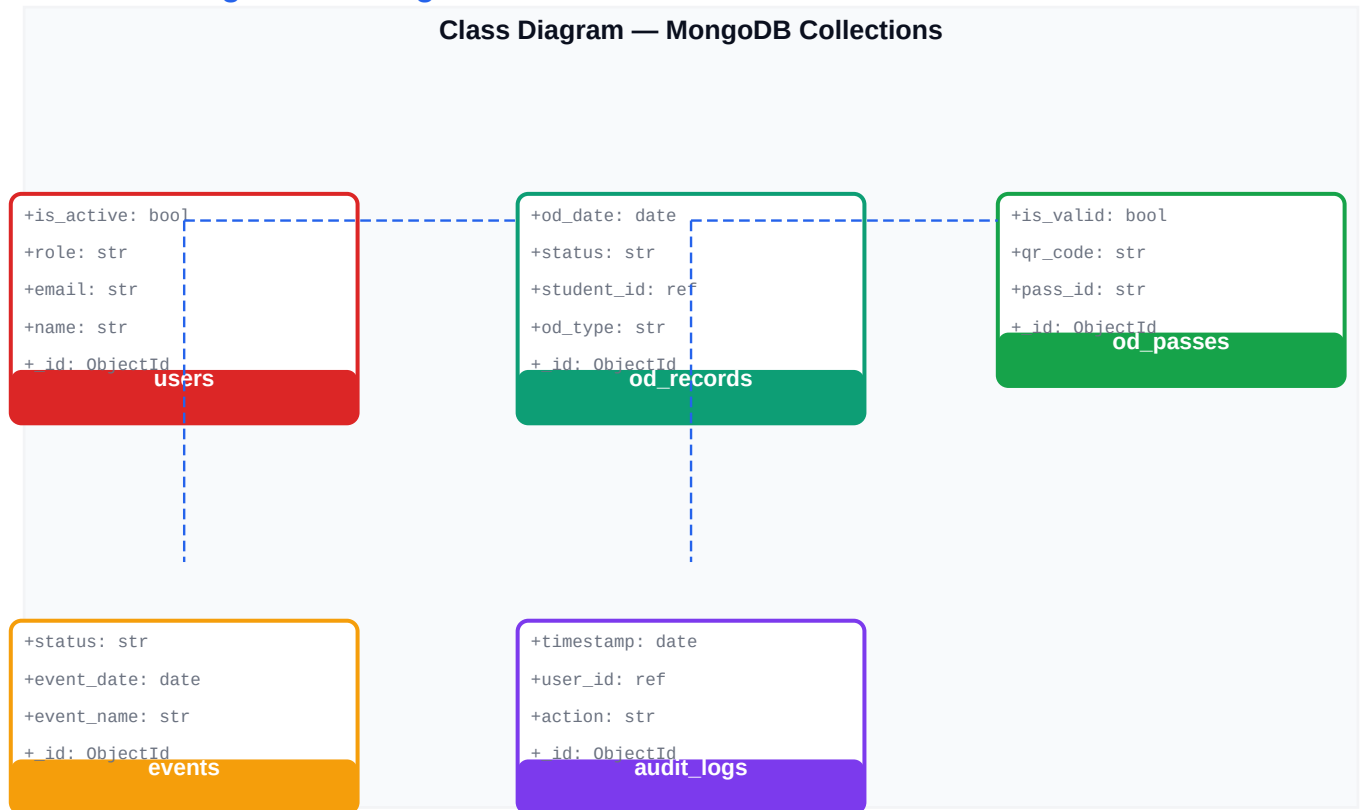
Six processes handle the data flows: P1 (User Management), P2 (OD Submission), P3 (Individual OD), P4 (Final Approval), P5 (First-Level Approval), P6 (OD Verification). All processes read/write to the central MongoDB data store.

6.5 Sequence Diagram — OD Approval Flow



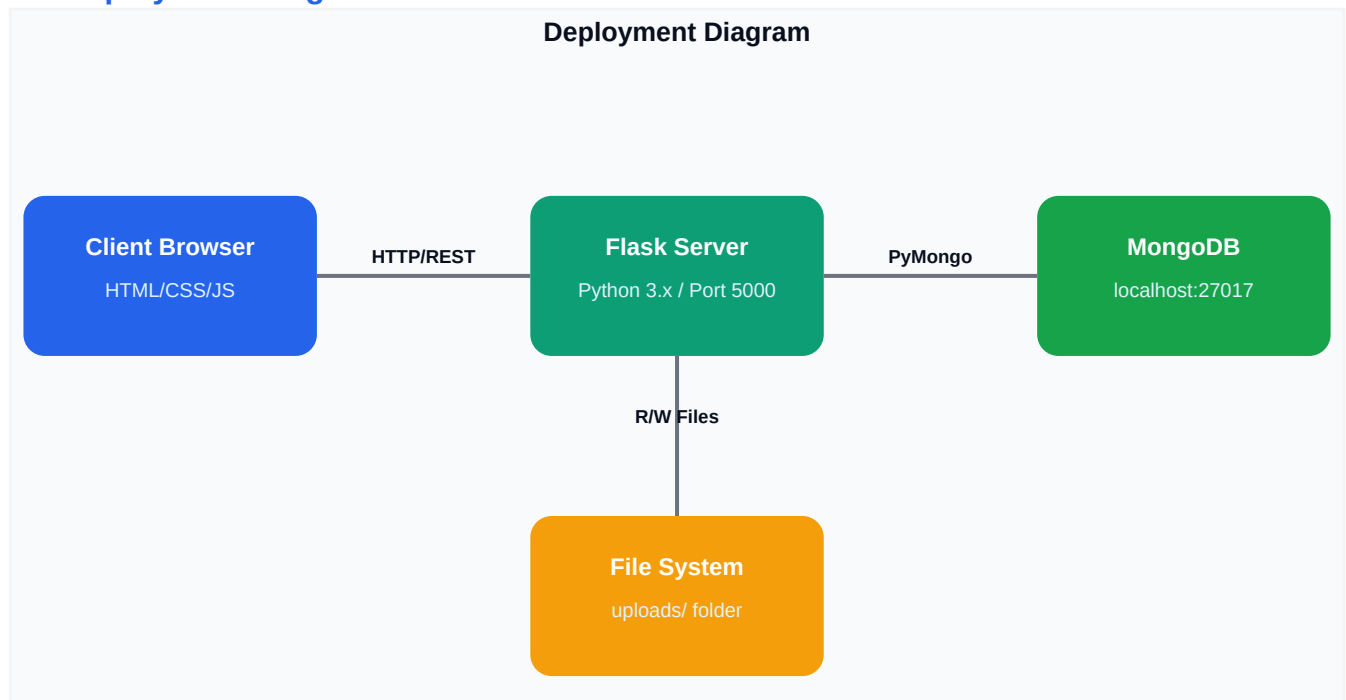
The sequence illustrates the end-to-end approval flow: Student/Event Head submits request → System notifies Department Head → Department Head approves → HOD is notified → HOD gives final approval → System generates OD Pass → Student is notified → Faculty verifies.

6.6 Class Diagram — MongoDB Collections



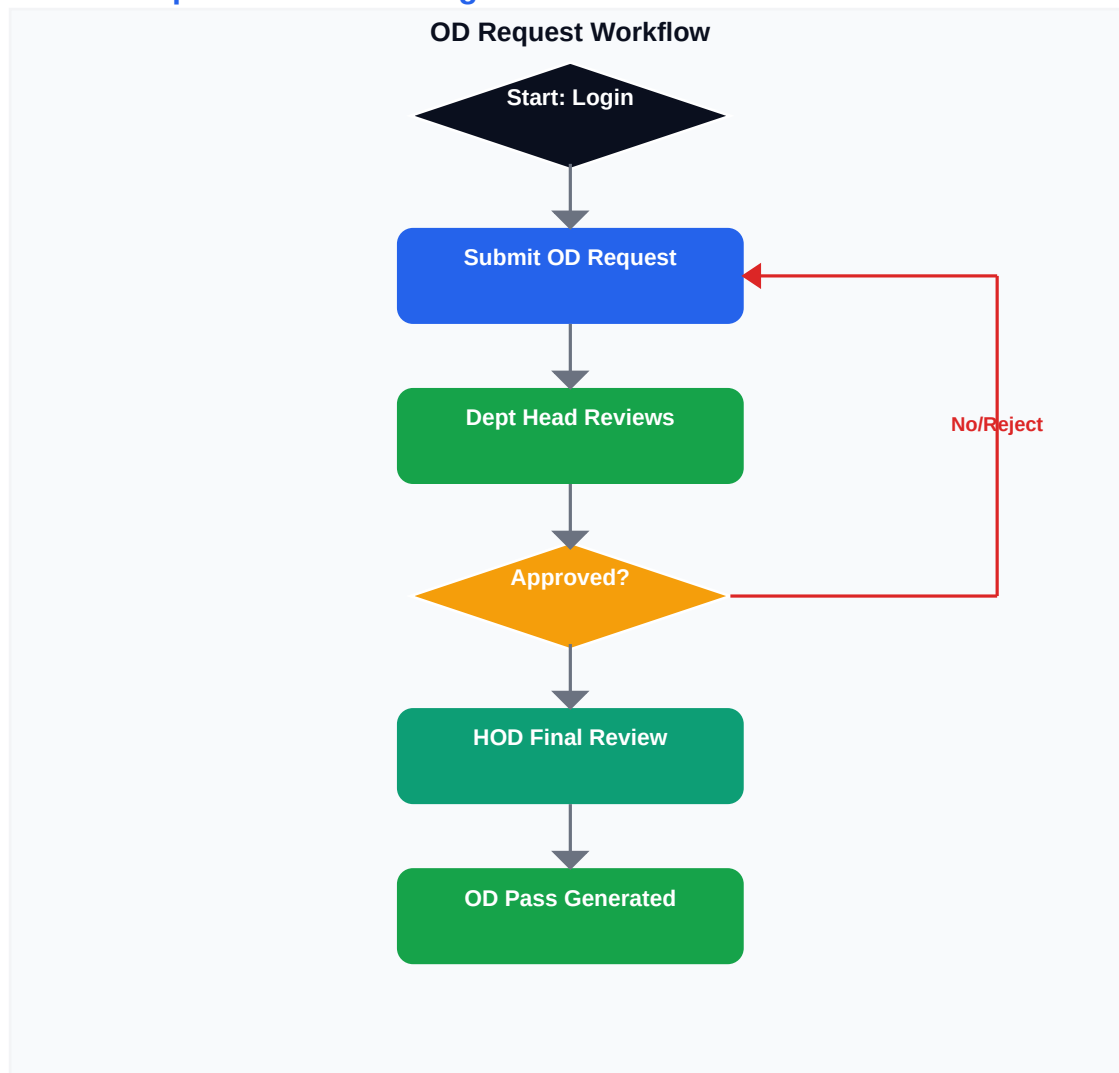
Core collections with their primary fields. *users* is the central entity; *od_records* references *users* (as student) and events. *od_passes* are generated from *od_records*. *audit_logs* reference *users* for traceability.

6.7 Deployment Diagram



Client browser communicates with the Flask server over HTTP/REST. Flask connects to MongoDB via PyMongo. File uploads (OD documents, HOD letters) are stored on the local file system under the uploads/ directory.

6.8 OD Request Workflow Diagram



Complete OD lifecycle: Login → Submit Request → Department Head Review → Approval check (rejection loops back to Event Head/Student) → HOD Final Review → OD Pass Generated.

7. Module Description

Admin Module

The Admin module is the backbone of user management. The system administrator (typically the IT department) creates all user accounts — there is no public self-registration. Students are bulk-uploaded via an Excel file with columns: roll_no, name, department, year, dob. Their default password is set to the DOB (DDMMYYYY format). Staff are uploaded similarly using: staff_id, name, email, department, role. The Admin can activate/deactivate accounts, reset passwords, and view all users filtered by role.

HOD Module

The HOD (Head of Department) module provides final authority in the OD approval chain. The HOD sees all Department-Head-approved OD records and must upload an official HOD approval letter (PDF) before the Approve button activates. The system includes a Bulk Approve option for high-volume periods. A red conflict alert banner warns the HOD if the same student has two OD requests on the same date. Upon final approval, the system auto-generates a unique OD Pass in the format OD-[YEAR]-[DEPT]-[0001] and triggers QR code generation via the qrcode library.

Department Head Module

The Department Head module handles the first level of OD review. It displays pending OD requests from both Event Heads (Event OD) and Students (Individual OD). Each record shows an OD Type badge, student details, event/reason, timing, and a View Proof button that opens the supporting document in a modal. The Department Head can approve individually or in bulk, and must provide a written reason before rejecting. Approved records are automatically forwarded to the HOD queue.

Event Head Module

The Event Head module allows authorized staff to create events and submit lists of participating students for OD. Students are added by roll number search (with auto-fill of name and department). Once the list is submitted to the Department Head, it is locked — no further edits are permitted. If the request is rejected by the HOD, the Event Head can edit and resubmit. Each student's OD status badge (Pending / Approved / Rejected) is visible at a glance.

Faculty Module

The Faculty (Teacher) module is read-only. Its primary feature is a large roll-number search bar. The result card displays: student name, department, OD type badge, event name or individual reason, OD date and timing, two approval status badges (Department Head and HOD), the OD Pass ID, and buttons to view the full OD Pass and HOD letter. A Class-wise OD Summary table shows, for today's date, how many students are on OD per period — with a modal listing their names and roll numbers.

Student Portal Module

Students can log in to apply for Individual OD (for reasons such as internships, medical appointments, or court appearances) by filling a form with reason, date, start/end time, and an optional supporting document (PDF/JPG/PNG, max 5 MB). A horizontal four-step timeline tracker (Submitted → Dept Head Review → HOD Review → Pass Generated) shows real-time approval status. Once approved, the student can view a

physical-ID-card-styled OD Pass and download it as a PDF containing a QR code. If rejected, the reason is displayed and the student may reapply up to two times.

8. MongoDB Collections & Schema

users

Field	Type / Description
_id	ObjectId
name	string
email	string
password	string (bcrypt)
role	admin/hod/dept_head/event_head/faculty/student
is_active	boolean
is_first_login	boolean
roll_no	string (students)
staff_id	string (staff)
department	string
academic_year	string

od_records

Field	Type / Description
_id	ObjectId
od_type	event / individual
student_id	ref → students
event_id	ref → events (nullable)
reason	string (Individual OD)
od_date	datetime
status	pending/dept_approved/hod_approved/rejected
supporting_document	string (filepath)
dept_rejection_reason	string
hod_rejection_reason	string
reapply_count	integer (max 2)
academic_year	string

od_passes

Field	Type / Description
_id	ObjectId
pass_id	string OD-2026-CS-0045
student_id	ref → students
od_record_id	ref → od_records
od_type	event / individual

qr_code	base64 PNG string
hod_letter_path	string (filepath)
is_valid	boolean

9. API Routes

Auth /api/auth

Method	Endpoint	Description
POST	/login	Credential + password → JWT token + role
POST	/logout	Invalidate token
POST	/change-password	First-login forced password change

Admin /api/admin

Method	Endpoint	Description
GET	/users	List all users (filter by role, status)
POST	/users	Create single user
POST	/users/bulk-upload	Excel upload → bulk user creation
PUT	/users/:id	Edit user details
DELETE	/users/:id	Deactivate account
GET	/template/students	Download student Excel template
GET	/template/staff	Download staff Excel template

Department Head /api/dept-head

Method	Endpoint	Description
GET	/pending	All pending OD for this department
POST	/approve/:id	First-level approve
POST	/approve/bulk	Bulk first-level approve
POST	/reject/:id	Reject with reason

HOD /api/hod

Method	Endpoint	Description
GET	/pending	All dept_approved OD records
POST	/approve/:id	Final approve + trigger pass generation
POST	/approve/bulk	Bulk final approve
POST	/upload-letter/:id	Upload HOD PDF letter
GET	/analytics	Department & event-wise stats
GET	/audit-logs	Full system audit log
GET	/export	Download PDF/Excel report

Faculty /api/faculty

Method	Endpoint	Description
GET	/search?roll_no=	Search student OD by roll number
GET	/class-summary	Today's period-wise OD count

Student /api/student

Method	Endpoint	Description
POST	/individual-od	Apply individual OD with document
POST	/individual-od/reapply/:id	Reapply (max 2 attempts)
GET	/od-pass/:id/download	Download OD Pass as PDF

10. Implementation — Screenshots

The following pages contain screenshots of the PassProof web application. Placeholder boxes are provided below — replace them with actual screenshots taken from <http://127.0.0.1:5000> before final submission.

[Screenshot Placeholder]
Login Page — <http://127.0.0.1:5000/login>

Fig: Login Page — <http://127.0.0.1:5000/login>

[Screenshot Placeholder]
Admin Dashboard — User Management Table

Fig: Admin Dashboard — User Management Table

[Screenshot Placeholder]
Admin — Bulk Upload with Excel Template Download

Fig: Admin — Bulk Upload with Excel Template Download

[Screenshot Placeholder]

Department Head Dashboard — Pending OD with View Proof

Fig: Department Head Dashboard — Pending OD with View Proof

[Screenshot Placeholder]

HOD Dashboard — Conflict Alert & Bulk Approve

Fig: HOD Dashboard — Conflict Alert & Bulk Approve

[Screenshot Placeholder]

Event Head — Manage Students (Locked after Submit)

Fig: Event Head — Manage Students (Locked after Submit)

[Screenshot Placeholder]

Faculty Dashboard — Roll Number Search Result

Fig: Faculty Dashboard — Roll Number Search Result

[Screenshot Placeholder]

Student Portal — OD Pass Card with QR Code

Fig: Student Portal — OD Pass Card with QR Code

[Screenshot Placeholder]

Analytics Dashboard — Charts & Leaderboard

Fig: Analytics Dashboard — Charts & Leaderboard

[Screenshot Placeholder]

Audit Log — Timeline Feed with Color-coded Actions

Fig: Audit Log — Timeline Feed with Color-coded Actions

11. Backend Code Snippets

11.1 Flask App Entry Point (app.py)

```
from flask import Flask from flask_pymongo import PyMongo from flask_jwt_extended import
JWTManager from flask_cors import CORS import config, os app = Flask(__name__)
app.config["MONGO_URI"] = config.MONGO_URI + config.DB_NAME app.config["JWT_SECRET_KEY"] =
config.JWT_SECRET_KEY app.config["JWT_ACCESS_TOKEN_EXPIRES"] = config.JWT_ACCESS_TOKEN_EXPIRES
mongo = PyMongo(app) jwt = JWTManager(app) CORS(app) os.makedirs("uploads/od_documents",
exist_ok=True) os.makedirs("uploads/hod_letters", exist_ok=True) from routes.auth import auth_bp
from routes.admin import admin_bp from routes.hod import hod_bp from routes.dept_head import
dept_head_bp from routes.event_head import event_head_bp from routes.faculty import faculty_bp
from routes.student import student_bp app.register_blueprint(auth_bp, url_prefix="/api/auth")
app.register_blueprint(admin_bp, url_prefix="/api/admin") app.register_blueprint(hod_bp,
url_prefix="/api/hod") app.register_blueprint(dept_head_bp, url_prefix="/api/dept-head")
app.register_blueprint(event_head_bp, url_prefix="/api/event-head")
app.register_blueprint(faculty_bp, url_prefix="/api/faculty") app.register_blueprint(student_bp,
url_prefix="/api/student") if __name__ == "__main__": app.run(debug=True, port=5000)
```

11.2 Login Route — Auto Role Detection (routes/auth.py)

```
from flask import Blueprint, request, jsonify from flask_jwt_extended import create_access_token
import bcrypt from app import mongo auth_bp = Blueprint("auth", __name__) @auth_bp.route("/login",
methods=["POST"]) def login(): data = request.get_json() credential = data.get("credential",
"").strip() password = data.get("password", "") user = mongo.db.users.find_one({ "$or": [
{"email": credential}, {"roll_no": credential}, {"staff_id": credential}, ] }) if not user: return
jsonify({"error": "Invalid credentials"}), 401 if not user.get("is_active", True): return
jsonify({"error": "Account deactivated"}), 403 if not bcrypt.checkpw(password.encode(),
user["password"].encode()): return jsonify({"error": "Invalid credentials"}), 401 token =
create_access_token(identity=str(user["_id"]), additional_claims={"role": user["role"]}) return
jsonify({ "token": token, "role": user["role"], "is_first_login": user.get("is_first_login",
False) }), 200
```

11.3 QR Code Generator (utils/qr_generator.py)

```
import qrcode, io, base64 def generate_qr_code(pass_id: str) -> str: qr = qrcode.QRCode(
version=1, error_correction=qrcode.constants.ERROR_CORRECT_H, box_size=10, border=4, )
qr.add_data(f"http://localhost:5000/verify/{pass_id}") qr.make(fit=True) img =
qr.make_image(fill_color="black", back_color="white") buffer = io.BytesIO() img.save(buffer,
format="PNG") buffer.seek(0) b64 = base64.b64encode(buffer.getvalue()).decode() return
f"data:image/png;base64,{b64}"
```

11.4 OD Pass ID Generator (utils/pass_generator.py)

```
from app import mongo from datetime import datetime def generate_pass_id(department: str, year:
int = None) -> str: if year is None: year = datetime.now().year dept_code = "".join([w[0] for w in
department.split()][:4].upper()) last = mongo.db.od_passes.find_one( {"pass_id": {"$regex":
f"^[OD-]{year}-{dept_code}-"}}, sort=[("pass_id", -1)] ) if last: seq =
int(last["pass_id"].split("-")[-1]) + 1 else: seq = 1 return f"OD-{year}-{dept_code}-{seq:04d}"
```

12. Security Features

JWT Authentication	Every protected route requires a valid JWT token (8-hour expiry). The token encodes the user's role and ID. Flask-JWT-Extended validates tokens on every request via role-specific decorators (@hod_required, @faculty_required, etc.).
bcrypt Password Hashing	All passwords are hashed using bcrypt with a cost factor of 12 before storage. Plain-text passwords are never stored in MongoDB. Default credentials (DOB / Staff ID) are hashed at the time of account creation.
Role-Based Access Control (RBAC)	Each Flask blueprint is guarded by a role decorator. For example, the HOD blueprint rejects requests from non-HOD tokens with HTTP 403. Students can only read their own OD records; Event Heads can only manage their own events.
OD List Lock	Once an Event Head submits an OD list to the Department Head, the system sets a 'locked' flag. Any further edit attempts return HTTP 403. The lock is lifted only if the HOD rejects the request, allowing resubmission.
Conflict Detection	Before saving any OD record, the system queries MongoDB for overlapping OD dates for the same student. Detected conflicts are surfaced as red alert banners to the approving authority, who must manually resolve them before proceeding.
Secure File Serving	Uploaded documents (supporting proofs, HOD letters) are served via JWT-authenticated Flask routes. Filenames are sanitised with werkzeug.utils.secure_filename and prefixed with the student's roll number and a timestamp to prevent collision and path traversal.
Session Timeout	JWT tokens expire after 8 hours. A 15-minute warning banner appears in the UI before expiry. On expiry, the frontend automatically redirects to the login page, preventing unauthorised continued access on shared devices.
Audit Trail	Every create, update, delete, approve, and reject action is written to the audit_logs collection with: user ID, role, IP address, action description, target document ID, and timestamp. Audit logs are immutable — no update or delete routes exist for them.

13. Testing

13.1 Unit Testing

Individual route handlers and utility functions were tested in isolation using Python's unittest framework. Mock MongoDB collections were used to decouple tests from the live database.

13.2 Integration Testing

The complete approval workflow was tested end-to-end using Postman: Event Head submits OD → Dept Head approves → HOD uploads letter and approves → OD Pass generated → Faculty searches roll number and views pass. All API responses and status codes were verified at each step.

13.3 Test Cases

TC#	Test Case	Input	Expected Output	Status
TC01	Valid Login	Correct credential + password	JWT token + role returned	PASS
TC02	Invalid Login	Wrong password	401 + error toast	PASS
TC03	Bulk Student Upload	Valid .xlsx file	Accounts created, success count returned	PASS
TC04	Duplicate Roll No	Upload with existing roll	Row skipped, reported in error list	PASS
TC05	Submit OD List	Event Head submits list	Status → submitted, Dept Head notified	PASS
TC06	Conflict Detection	Same student, same date	Red alert banner shown	PASS
TC07	Dept Head Approve	Valid OD record	Status → dept_approved, HOD notified	PASS
TC08	HOD Approve w/o Letter	No letter uploaded	Approve button disabled	PASS
TC09	HOD Final Approve	Letter uploaded + clicked	OD Pass generated with QR	PASS
TC10	Faculty Roll Search	Valid roll number	Full OD details + pass displayed	PASS
TC11	Student OD Pass PDF	Approved OD	PDF downloaded with QR code	PASS
TC12	Rejection Reason	HOD rejects with reason	Reason visible in student portal	PASS
TC13	Reapply after Reject	Student reapplies	New OD record created (count +1)	PASS
TC14	Session Timeout	Token expired	Auto-logout + redirect to login	PASS
TC15	QR Code Scan	Scan OD Pass QR	Verification page shows student details	PASS

14. Conclusion

PassProof successfully demonstrates the application of modern full-stack web technologies to solve a practical, real-world administrative challenge in college environments. By introducing a structured, role-based, multi-level OD approval workflow, the system eliminates fake OD claims, reduces administrative burden, and brings full transparency to all stakeholders.

The project showcases key software engineering principles: separation of concerns (Flask blueprints), security best practices (JWT, bcrypt, RBAC), data integrity (conflict detection, list locking), and user experience design (responsive UI, toast notifications, skeleton loaders). The QR-code-based OD Pass verification adds a modern, tamper-proof layer of identity confirmation that sets this system apart from conventional attendance tools.

Future enhancements could include: integration with the college's Student Information System (SIS) for automatic student import, real-time push notifications via WebSockets, mobile app wrappers for Android/iOS, and biometric or face-recognition-based pass verification at event venues.

15. References

- [1] Flask Documentation — <https://flask.palletsprojects.com>
- [2] MongoDB Documentation — <https://www.mongodb.com/docs>
- [3] Flask-JWT-Extended — <https://flask-jwt-extended.readthedocs.io>
- [4] PyMongo Documentation — <https://pymongo.readthedocs.io>
- [5] ReportLab User Guide — <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- [6] openpyxl Documentation — <https://openpyxl.readthedocs.io>
- [7] qrcode Library — <https://pypi.org/project/qrcode/>
- [8] Chart.js Documentation — <https://www.chartjs.org/docs>
- [9] W3Schools — HTML, CSS, JavaScript Tutorials — <https://www.w3schools.com>
- [10] GeeksforGeeks — Python Flask & MongoDB — <https://www.geeksforgeeks.org>
- [11] MDN Web Docs — Fetch API — <https://developer.mozilla.org>
- [12] TailwindCSS (reference for design tokens) — <https://tailwindcss.com>